DAY6 - Modernization Strategies & Migration Planning

- Migration phases: assess, containerize, deploy, modernize
- Lift & Shift vs Replatform vs Refactor
- Application dependency mapping & tooling
- Hybrid & multi-cloud patterns
- Hands-On: Create migration plan for sample on-prem workload

Modernization means:

Improving an application using cloud-native technologies to make it scalable, reliable, and efficient.

Legacy - Application lets say python 2.7

Language: Python 2.7 ❌ (End of Life)
Runs on: VM (Ubuntu)
Framework: Flask/Django (old version)
Database: MySQL
Deployment: Manual (SSH + run script)

Problems with Python 2.7 (Legacy):

1. No Support
- Python 2.7 is EOL (End of Life)
- Python 2.7 is EOL (End of Life)

2. Security Risks
- Vulnerable to attacks
- No updates

3. Not Cloud-Friendly
- Hard to containerize properly
- Dependency conflicts

4. No Scalability
- Runs on single VM
- Cannot handle traffic spikes

5. Manual Deployment
- SSH into server
- Run commands manually

Suitable for the business ??

1. Monolithic Application
- Entire banking app in one codebase
- Login, payments, transfers → tightly coupled

2. Runs on VMs
- Fixed capacity
- Cannot handle peak traffic

3. No Auto Scaling
- During salary day / festival:

Users ↑ → System crashes ❌

4. Manual Deployment
- Downtime during updates
- Risky releases

5. Security Risks
- Old runtimes (Python 2.7)
- No patching

Legacy Typical Banking System:

Internet Banking App
+
App Server (Python 2.7 / Java)
+
Database (Oracle / MySQL)
+
External APIs (Payment, KYC, OTP)

Goal:

Legacy Monolith App (VM)
+
Modern Cloud-Native App (GKE + Microservices)

** Modernization Target Architecture (GKE):

Users
↓
Load Balancer (Service)
↓
GKE Cluster
↓
Microservices (Pods)
↓
Managed DB (Cloud SQL)
↓
Monitoring & Logging

security features -> Load distribution + Incoming & Outgoing filter out

- Add the GKE Roles
- Storage , networking, Security
- Cloud Based database - Relational / Non-Relational
- Set up the alerts / Logs

Legacy App (VM)
↓
Containerize (Docker)
↓
Push to Registry
↓
Deploy to GKE
↓
Expose via LoadBalancer
↓
Enable Autoscaling (HPA)
↓
Connect Cloud SQL
↓
Add Monitoring & Logging
↓
Setup Alerts

User → LoadBalancer → GKE Pods → Cloud SQL
↓
Monitoring + Logs + Alerts

STEP-BY-STEP UI - FLOW :

Step1: Application --> Coding --> Containerize it -> IMAGE

Step 2: Create GKE Cluster

Step 3: Pull the Containerized Image

Step4: Deploy it in the GKE - WORKLoad

* Deploy Microservices:
1. Deploy auth-service
Fill:
Name: auth-service
Image: gcr.io/project/auth-service
Replicas: 2

It means : 1 Microservice as : (1) account-service

2nd 1 Microservice as (2)payment-service

Step5: Enable Autoscaling (HPA):

Step 6: Configure Database (Cloud SQL)

Step 7: Configure Environment Variables & Enable monitoring , Alerts

Step 8: then Deploy

Real Banking Mapping

Component	GCP Service
App	GKE
DB	Cloud SQL
Scaling	HPA
Logs	Cloud Logging
Monitoring	Cloud Monitoring

Use Case: University Result Management System:

Problem Statement:

A university publishes results online:

Thousands of students check results at the same time

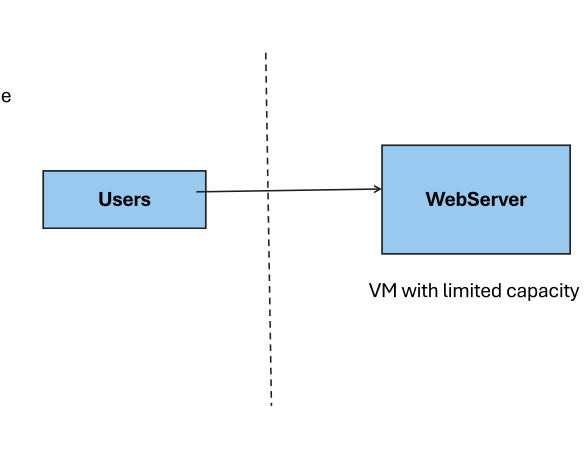
Especially on result day → heavy traffic spike

* Legacy System issues:

Single Web Server (VM)
↓
Application (Monolith – Python 2.7 / PHP)
↓
Database (MySQL)

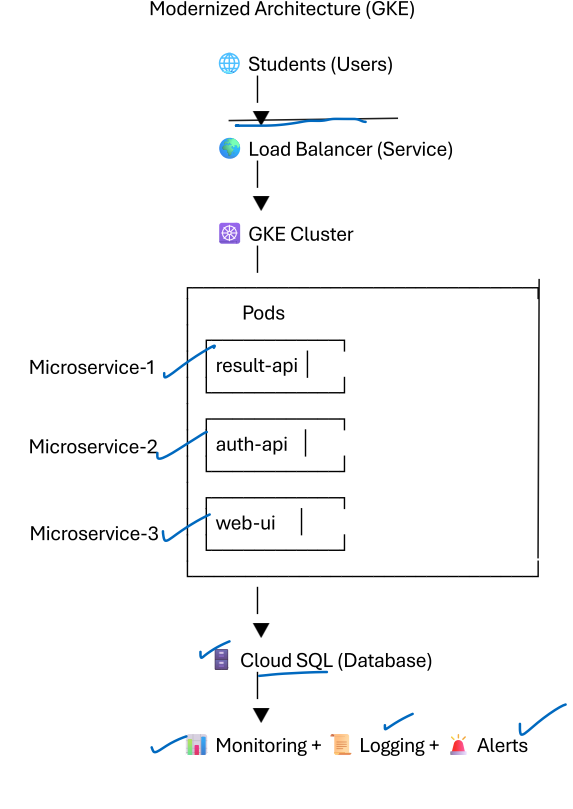
Problems:

- Server crashes on result day
- Slow response
- No scaling
- Manual deployment
- Downtime



VM with limited capacity

Modernized Architecture (GKE)



* Components Explanation:

🌐 1. Load Balancer
Entry point for students
Distributes traffic

🟪 2. GKE Cluster
Runs application containers
Auto-manages infrastructure

📦 3. Microservices

Service	Purpose
web-ui	Student interface
result-api	Fetch results
auth-api	Login/authentication

🗄 4. Cloud SQL - Autoscaling
- Stores student results
- Managed DB

📊 5. Observability
Monitoring → performance
Logging → errors
Alerts → failures

Feature	Legacy	Modern
Traffic handling	Poor	Excellent
Scaling	None	Auto
Downtime	High	Zero
Deployment	Manual	Automated

→ HPA

* Various Migration phases: assess, containerize, deploy, modernize:

1. ASSESS PHASE

Analyze current system, dependencies, risks, and readiness for cloud.

University Example

Current System:

App: PHP / Python 2.7
Server: Single VM
DB: MySQL
Users: 10,000 (result day spike)

* What You Identify

App type → Monolith
DB size → Large
Traffic → High spike
Risk → Server crash

* On result day:
All students login → server overload → crash ❌

Step 1: *-> Create Assessment Sheet:

App | Runtime | DB | Traffic | Risk

Step 2: Fill Data

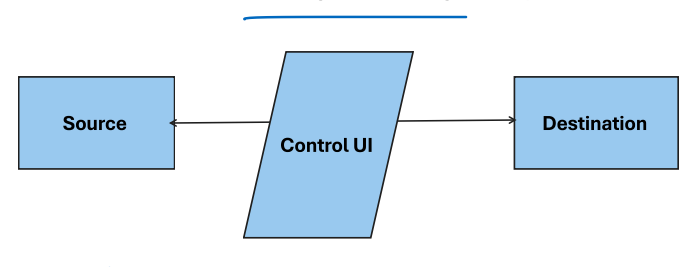
App	Runtime	DB	Traffic	Risk
Result App	Python 2.7	MySQL	High	High

Step 3: Draw Architecture:

User → App → DB

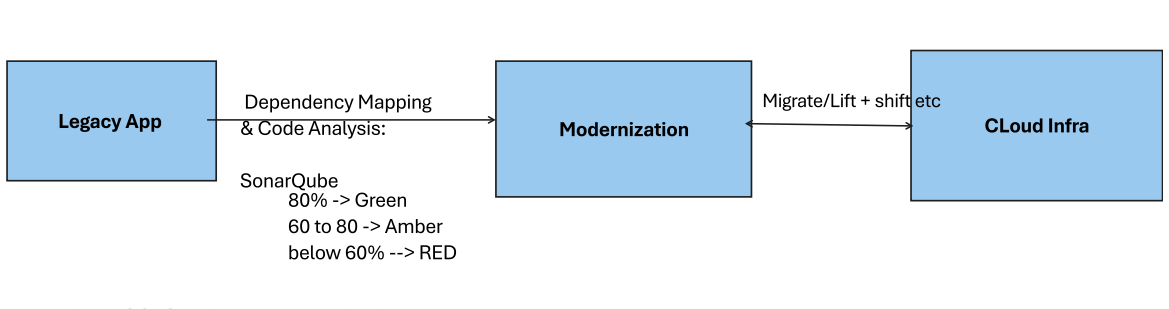
Outcome:
✓ Full understanding of system
✓ Migration plan ready

Tools:
CLoud Based : GCP - Migrate / AWS migrate Template



** Dependency Mapping **
- Dynatrace
- AppDynamics

** Code Analysis:
SonarQube: SonarQube is a comprehensive code quality and security analysis platform developed by SonarSource

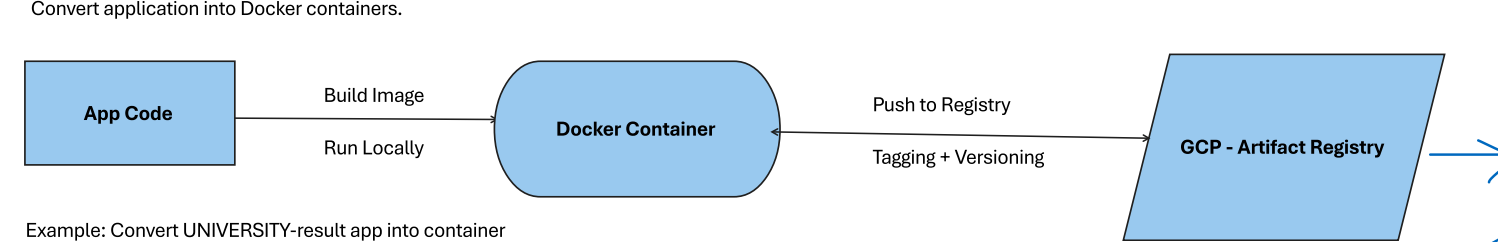


e.g: Micro FOCUS mainframe Modernization

GCP - Modernization & Migration

2. CONTAINERIZE PHASE

Convert application into Docker containers.



Outcome:

-> App becomes portable
->Ready for Kubernetes

Example: Convert UNIVERSITY-result app into container

Dockerfile Example:

```
FROM python:3.10
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

** As Industry Scenario:
University wants: -> Same app running everywhere, Also No dependency issues

3. DEPLOY PHASE (GKE)

Deploy containerized app to Kubernetes cluster.

Deploy result app to GKE

Step 1: Create Cluster
Step 2: Deploy App
Step 3: Expose App
Step 4: Access App

-> App live on cloud
-> Accessible globally

4 . MODERNIZE PHASE:
Enhance application using cloud-native features.

Example (Result System)

-> Add: Autoscaling, Monitoring, Microservices

* Modernization Features:
1. Autoscaling (HPA) ->
UI: Workloads → Scaling

Min: 2
Max: 10
CPU: 60%

Handles result day traffic

CUppose CPU 60% + above cross -> then new Pods will be created & managed

2. Monitoring:
Monitoring → Dashboard

Show: CPU, Pods

3. Logging: Logs Explorer
Query: resource.type="k8s_container"

4. Microservices:
Break app into: result-api, auth-service

** FULL PHASE COMPARISON **

Phase	What Happens	Output
Assess	Analyze system	Plan
Containerize	Create Docker image	Portable app
Deploy	Run on GKE	Live app
Modernize	Add cloud features	Scalable system

* Outcome
✓ High performance
✓ No downtime
✓ Scalable system

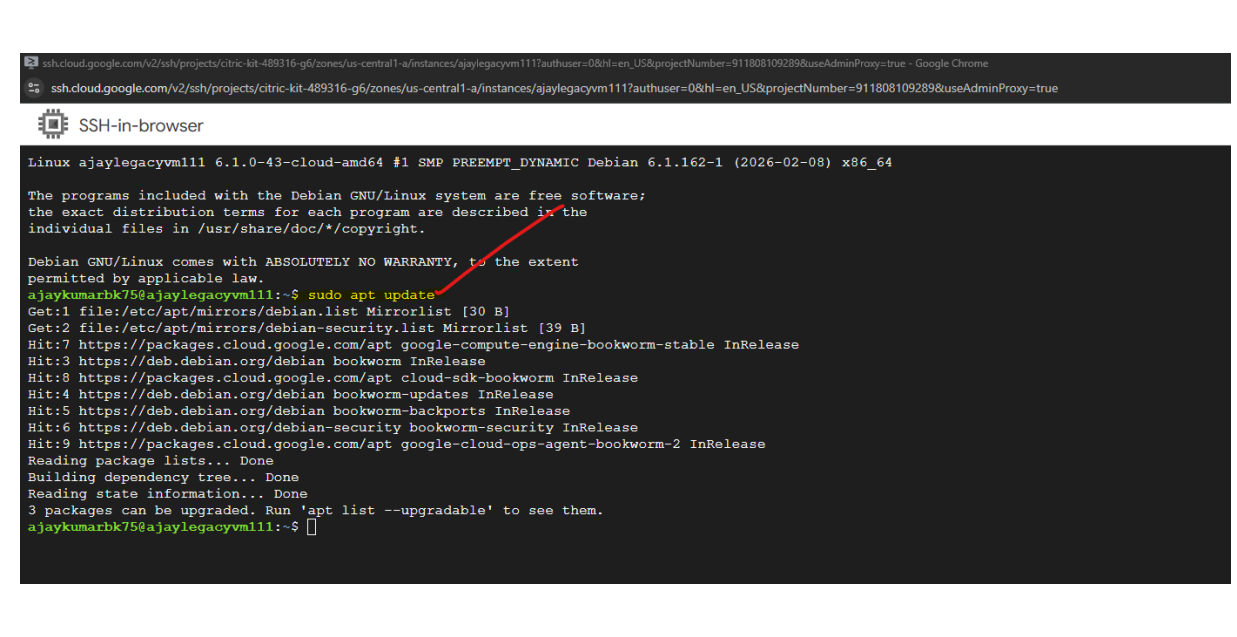
DEMO: Legacy application to GCP - GKE & Modernize it

Compute Engine (Legacy VM)
↓
Containerize inside VM
↓
Push to Artifact Registry
↓
Deploy to GKE

** Visual FLow Diagram:

VM (Legacy App)
↓
Dockerize
↓
Push to Artifact Registry
↓
Deploy to GKE
↓
Expose via LoadBalancer
↓
Enable Autoscaling
↓
Modern Cloud App ✅

Step 1: CREATE COMPUTE Engine VM



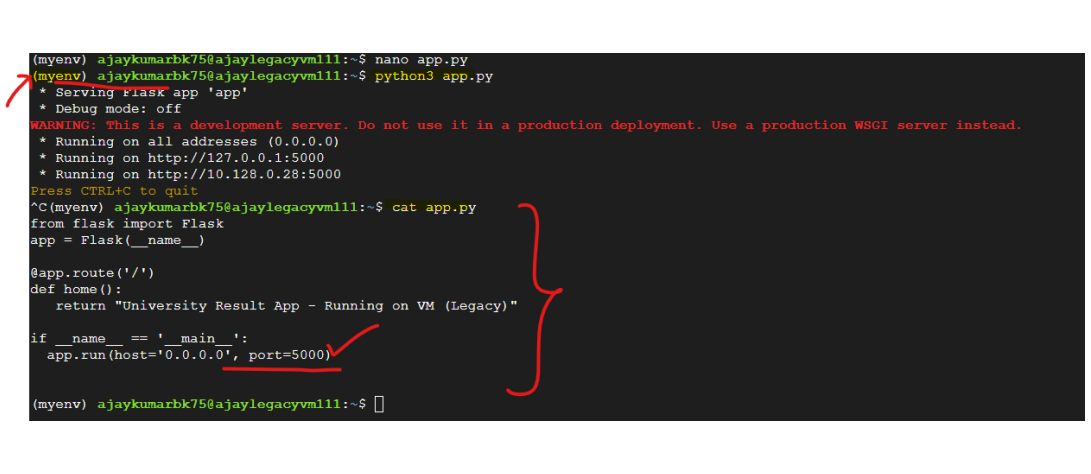
```
ssh.cloud.google.com/v2/ssh/projects/citric-kit-489316-g6/zones/us-central1-a/instances/ajaylegacyvm111?authuser=0&hl=en_US&projectNumber=911808109289&useAdminProxy=true
SSH-in-browser
Linux ajaylegacyvm111 6.1.0-43-cloud-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.162-1 (2026-02-08) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
ajaykumarbk75@ajaylegacyvm111:~$ sudo apt update
Get:1 file:/etc/apt/mirrors/debian.list Mirrorlist [30 B]
Get:2 file:/etc/apt/mirrors/debian-security.list Mirrorlist [39 B]
Hit:7 https://packages.cloud.google.com/apt google-compute-engine-bookworm-stable InRelease
Hit:3 https://deb.debian.org/debian bookworm InRelease
Hit:8 https://packages.cloud.google.com/apt cloud-sdk-bookworm InRelease
Hit:4 https://deb.debian.org/debian bookworm-updates InRelease
Hit:5 https://deb.debian.org/debian bookworm-backports InRelease
Hit:6 https://deb.debian.org/debian-security bookworm-security InRelease
Hit:9 https://packages.cloud.google.com/apt google-cloud-ops-agent-bookworm-2 InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
3 packages can be upgraded. Run 'apt list --upgradable' to see them.
ajaykumarbk75@ajaylegacyvm111:~$
```

Step 2: Host Legacy App on Compute Engine (VM)

sudo apt update
sudo apt install python3-pip -y

** Create virtual environment

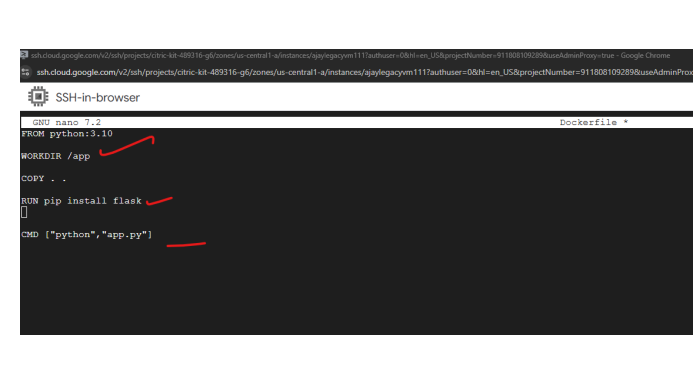
Running Flask app - Done



```
(myenv) ajaykumarbk75@ajaylegacyvm111:~$ nano app.py
(myenv) ajaykumarbk75@ajaylegacyvm111:~$ python3 app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.128.0.28:5000
Press CTRL+C to quit
^C(myenv) ajaykumarbk75@ajaylegacyvm111:~$ cat app.py
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return "University Result App - Running on VM (Legacy)"

if __name__ == '__main__':
  app.run(host='0.0.0.0', port=5000)

(myenv) ajaykumarbk75@ajaylegacyvm111:~$
```

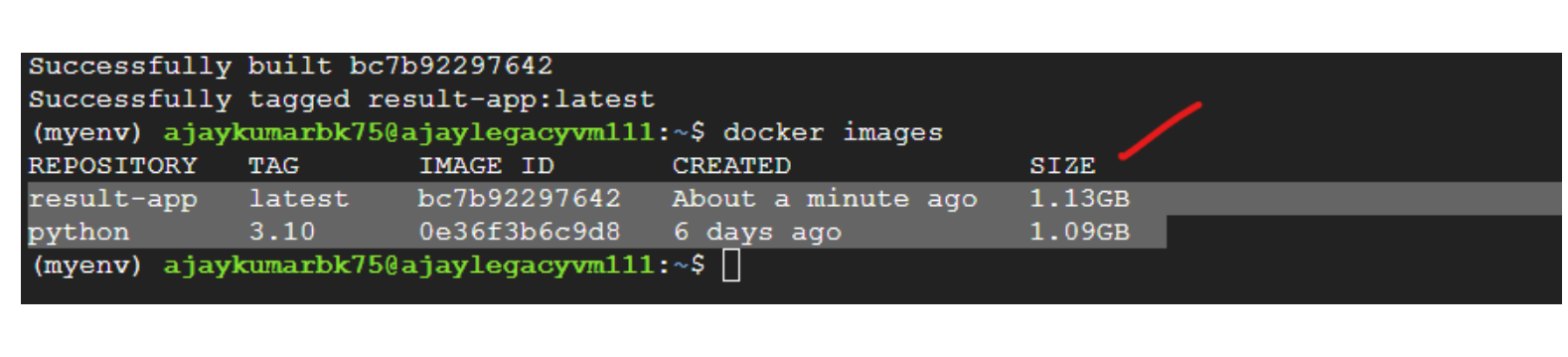
Step 3 : Containerize Inside VM

Install Docker & write docker file



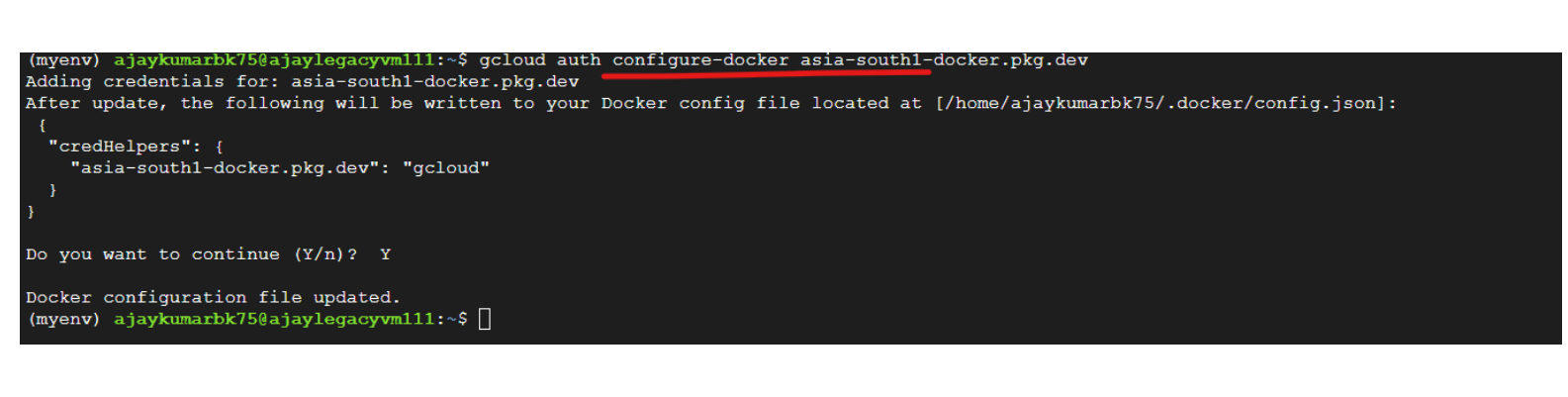
```
SSH-in-browser
  GNU nano 7.2                                   Dockerfile *
FROM python:3.10

WORKDIR /app

COPY . .

RUN pip install flask


CMD ["python","app.py"]
```

Build Docker Image & Run it



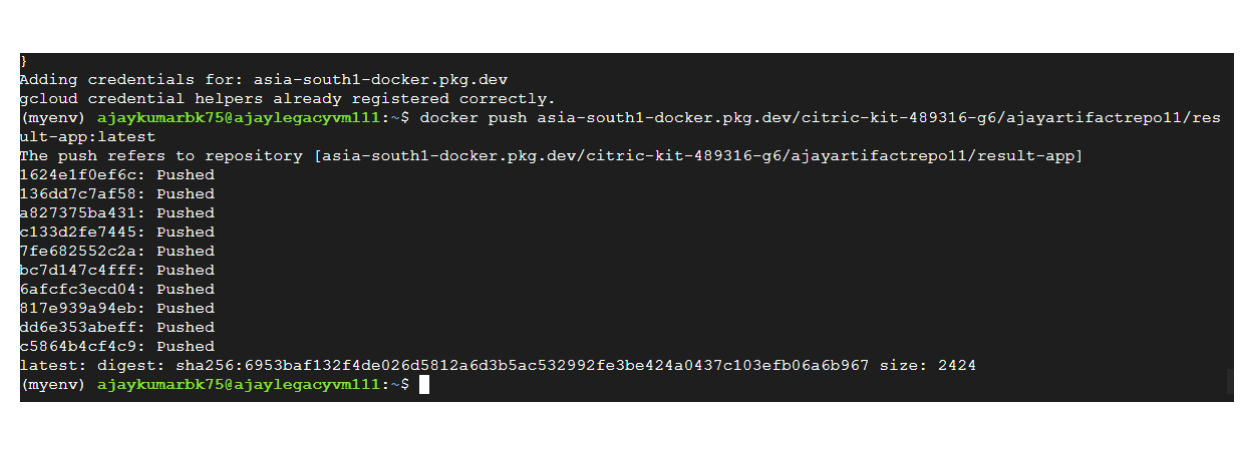
```
Successfully built bc7b92297642
Successfully tagged result-app:latest
(myenv) ajaykumarbk75@ajaylegacyvm111:~$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED            SIZE
result-app    latest    bc7b92297642   About a minute ago   1.13GB
python        3.10      0e36f3b6c9d8   6 days ago         1.09GB
(myenv) ajaykumarbk75@ajaylegacyvm111:~$
```

Step 4: Authenticate Docker with Artifact Registry

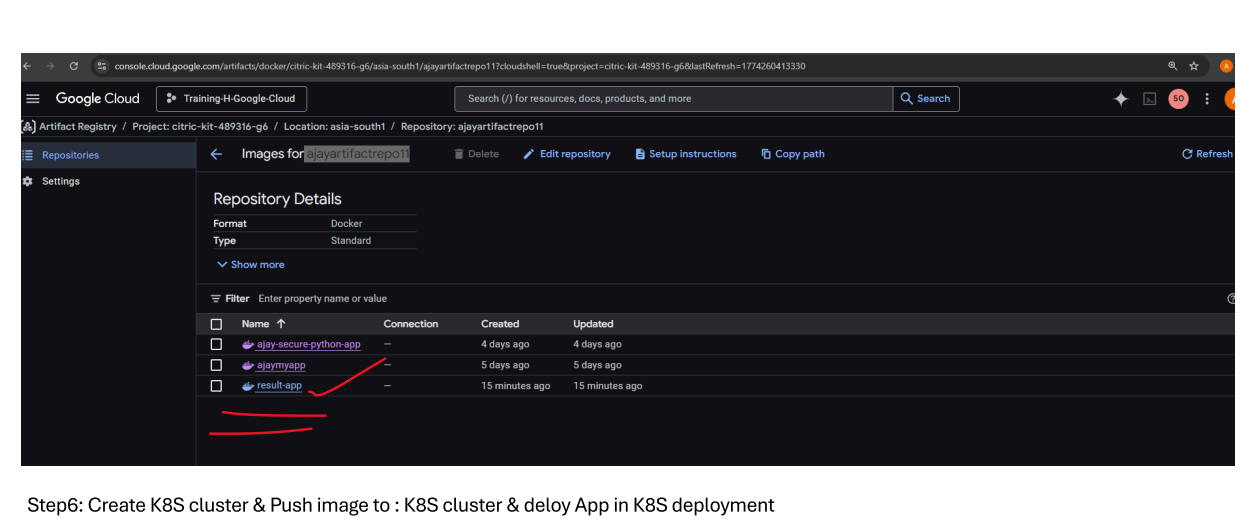


```
(myenv) ajaykumarbk75@ajaylegacyvm111:~$ gcloud auth configure-docker asia-south1-docker.pkg.dev
Adding credentials for: asia-south1-docker.pkg.dev
After update, the following will be written to your Docker config file located at [/home/ajaykumarbk75/.docker/config.json]:
 {
  "credHelpers": {
    "asia-south1-docker.pkg.dev": "gcloud"
  }
}

Do you want to continue (Y/n)?  Y

Docker configuration file updated.
(myenv) ajaykumarbk75@ajaylegacyvm111:~$
```

Step 5: Push the image into GCP Artifact registry



```
}
Adding credentials for: asia-south1-docker.pkg.dev
gcloud credential helpers already registered correctly.
(myenv) ajaykumarbk75@ajaylegacyvm111:~$ docker push asia-south1-docker.pkg.dev/citric-kit-489316-g6/ajayartifactrepo11/res
ult-app:latest
The push refers to repository [asia-south1-docker.pkg.dev/citric-kit-489316-g6/ajayartifactrepo11/result-app]
1624e1f0ef6c: Pushed
136dd7c7af58: Pushed
a827375ba431: Pushed
c133d2fe7445: Pushed
7fe682552c2a: Pushed
bc7d147c4fff: Pushed
6afcfc3ecd04: Pushed
817e939a94eb: Pushed
dd6e353abeff: Pushed
c5864b4cf4c9: Pushed
latest: digest: sha256:6953baf132f4de026d5812a6d3b5ac532992fe3be424a0437c103efb06a6b967 size: 2424
(myenv) ajaykumarbk75@ajaylegacyvm111:~$
```

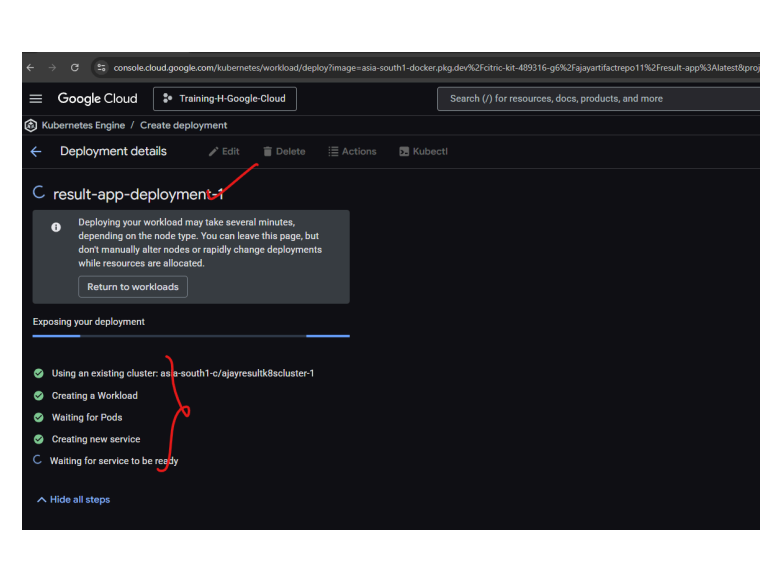


Artifact Registry / Project: citric-kit-489316-g6 / Location: asia-south1 / Repository: ajayartifactrepo11

Repository Details
Format: Docker
Type: Standard

Name ↑	Connection	Created	Updated
ajay-secure-python-app	–	4 days ago	4 days ago
ajaymyapp	–	5 days ago	5 days ago
result-app	–	15 minutes ago	15 minutes ago

Step6: Create K8S cluster & Push image to : K8S cluster & deloy App in K8S deployment



Kubernetes Engine / Create deployment

Deployment details

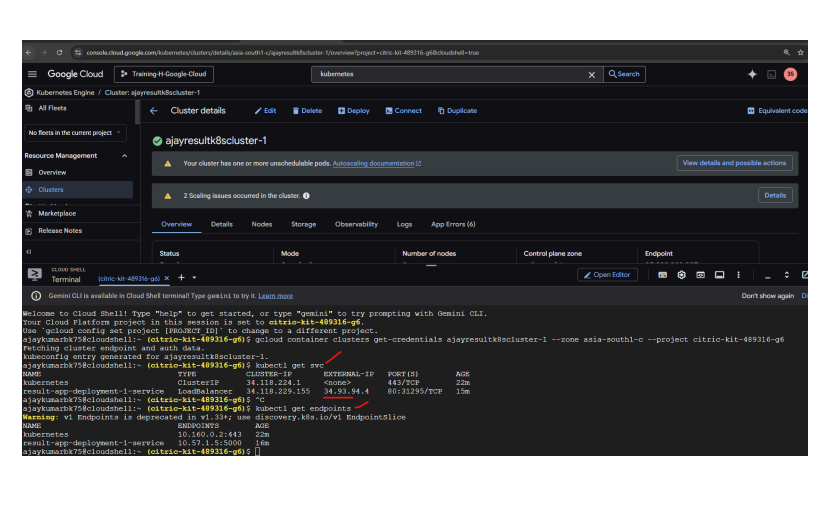
result-app-deployment-1

Deploying your workload may take several minutes, depending on the node type. You can leave this page, but don't manually alter nodes or rapidly change deployments while resources are allocated.

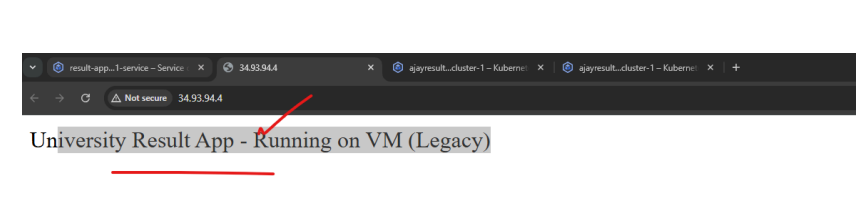
Exposing your deployment

- Using an existing cluster: asia-south1-c/ajayresultk8scluster-1
- Creating a Workload
- Waiting for Pods
- Creating new service
- Waiting for service to be ready

Step 7: Access the app from external end point



ajayresultk8scluster-1

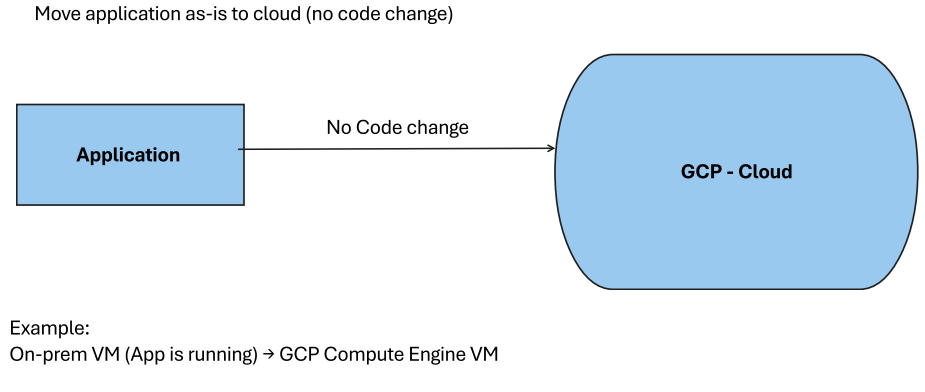


Not secure 34.93.94.4

University Result App - Running on VM (Legacy)

** Lift & Shift (Rehost)

Move application as-is to cloud (no code change)



Example:
On-prem VM (App is running) → GCP Compute Engine VM

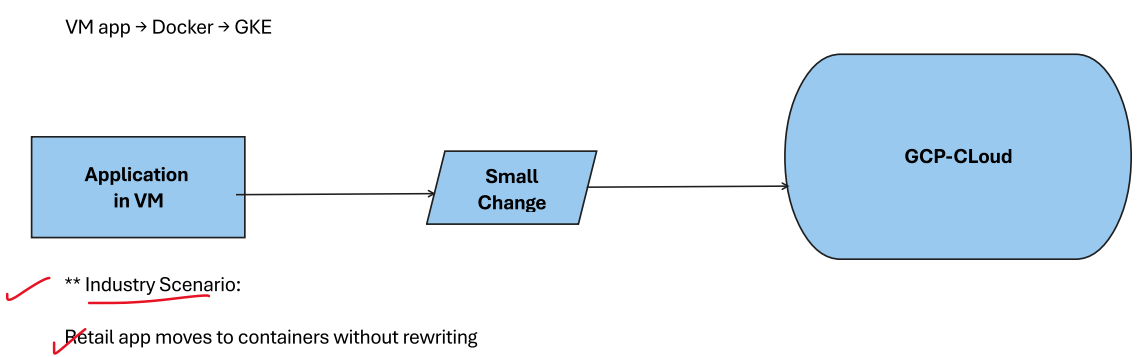
Industry Scenario:

Banking app migrated quickly due to deadline

** Replatform (Lift, Tinker & Shift)

Make small improvements (like containerization)

VM app → Docker → GKE



** Industry Scenario:

Retail app moves to containers without rewriting

** Refactor (Re-architect):

Redesign app to cloud-native architecture

Example:

Monolith → Microservices

Industry Scenario:

OTT platform splits services:

Auth
Streaming
Payments

Strategy	Effort	Change	Use Case
Lift & Shift	Low	None	Quick migration
Replatform	Medium	Small	Modernization
Refactor	High	Full redesign	Long-term

** Application dependency mapping & tooling

Application Dependency Mapping is the process of identifying how different components of an application (services, databases, APIs, OS libraries) are connected and depend on each other.

University Result App:

Frontend (Flask UI)
↓
Backend API
↓
Database (MySQL)
↓
External API (SMS / Email)

* If DB fails → App fails
* If API fails → Notification fails

** Real Use Case:

Banking App:

- Login service depends on:
- Auth service
- Database
- OTP API

During migration: If you miss OTP API → login breaks

Tools

1. Dynatrace → live dependency flow
2. AppDynamics → service mapping
3. SonarQube → code-level dependency

** Dependency Mapping for Flask App **

You have:

- Flask app
- SQLite DB
- External API

Identify dependencies

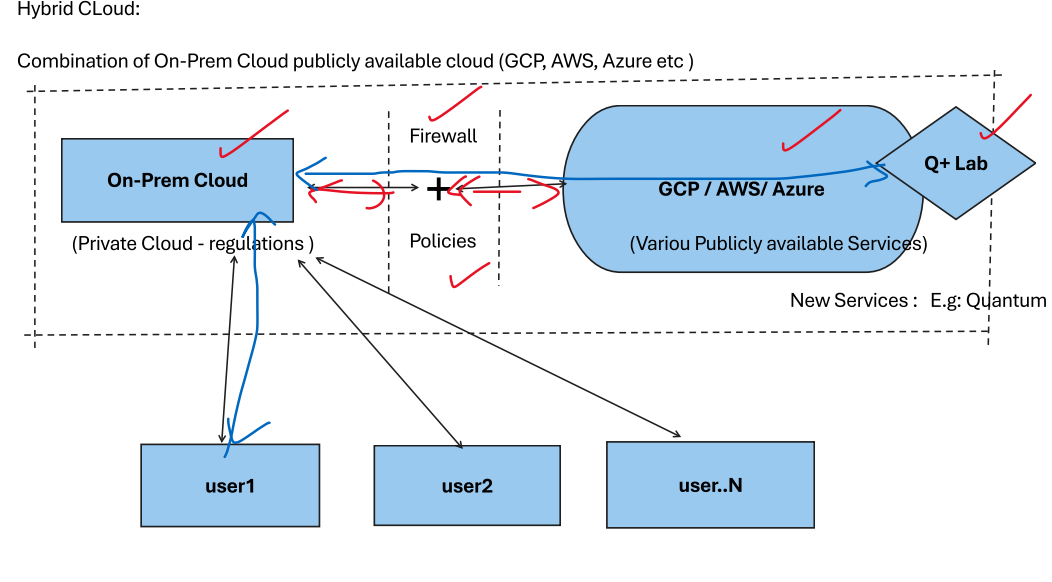
✓ Python
✓ Flask
✓ SQLite
✓ OS libraries

Dependency diagram
List of required components

Hybrid & Multi-Cloud Patterns

Hybrid CLoud:

Combination of On-Prem Cloud publicly available cloud (GCP, AWS, Azure etc)



New Services : E.g: Quantum Computing

Workloads are split based on:

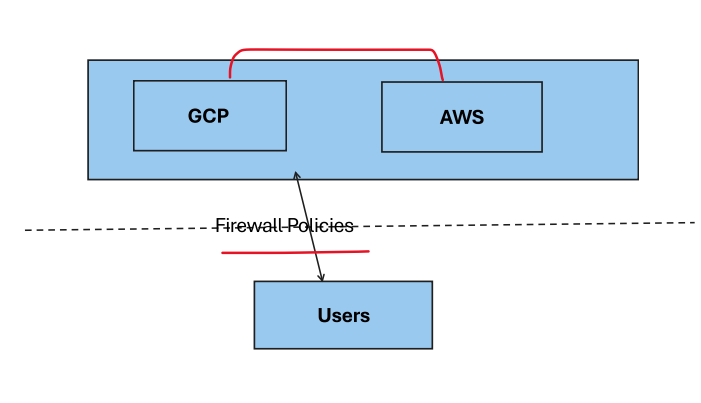
Security
Latency
Compliance

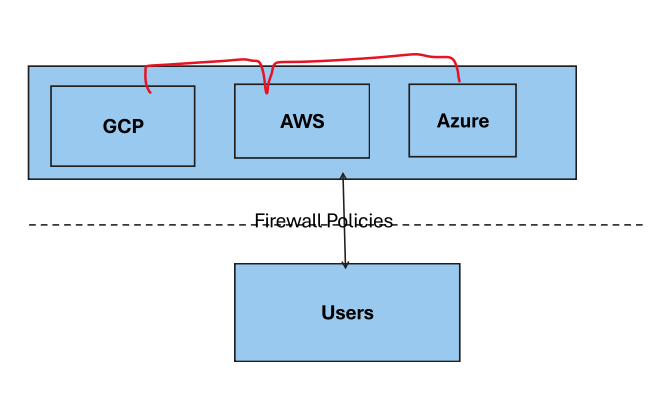
Multi-Cloud:

Using more than one cloud provider simultaneously

E.g:

GCP + AWS
GCP + Azure
AWS + Azure + GCP





E.g:
Frontend → GCP (GKE)

Backend → AWS (EC2 / EKS)

Database → Azure SQL

** Example: SBI / HDFC-like Architecture

Core Banking System → On-Prem (Highly secure)
+
API Layer → Hybrid Connection
+
Frontend / Mobile App → GCP (GKE)